

Algoritmos de Ordenação Polimórfica

Guia Prático e Comparativo em Java ($O(n^2)$ e $O(n \log n)$)

DOMÍNIO Hierarquia Abstrata de Mídias

```
abstract class Midia implements Comparable<Midia> {
    private String titulo;
    public Midia(String t) { this.titulo = t; }
    public String getTitulo() { return titulo; }
    public abstract double getMétricaOrdenacao();

    @Override
    public int compareTo(Midia outra) {
        return Double.compare(outra.getMétricaOrdenacao(), this.getMétricaOrdenacao());
    }
}

class Musica extends Midia {
    private int reproducoes;
    public Musica(String t, int r) { super(t); this.reproducoes = r; }
    @Override public double getMétricaOrdenacao() { return reproducoes; }
}

class Video extends Midia {
    private int views, curtidas;
    public Video(String t, int v, int c) { super(t); this.views = v; this.curtidas = c; }
    @Override public double getMétricaOrdenacao() { return views + (curtidas * 2.0); }
}
```

$O(N^2)$ 1. Bubble Sort & 2. Selection Sort

```
// 1. BUBBLE SORT - Compara e troca pares adjacentes
public static <T extends Comparable<T>> void bubbleSort(List<T> lista) {
    int n = lista.size();
    for (int i = 0; i < n - 1; i++) {
        boolean trocou = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (lista.get(j).compareTo(lista.get(j + 1)) > 0) {
                trocar(lista, j, j + 1);
                trocou = true;
            }
        }
        if (!trocou) break;
    }
}

// 2. SELECTION SORT - Busca o menor elemento e coloca no início
public static <T extends Comparable<T>> void selectionSort(List<T> lista) {
    int n = lista.size();
    for (int i = 0; i < n - 1; i++) {
        int menor = i;
        for (int j = i + 1; j < n; j++) {
            if (lista.get(j).compareTo(lista.get(menor)) < 0) menor = j;
        }
        if (menor != i) trocar(lista, i, menor);
    }
}
```

```
// 3. INSERTION SORT - Insere na posição correta da sublista ordenada
public static <T extends Comparable<T>> void insertionSort(List<T> lista) {
    for (int i = 1; i < lista.size(); i++) {
        T chave = lista.get(i);
        int j = i - 1;
        while (j >= 0 && lista.get(j).compareTo(chave) > 0) {
            lista.set(j + 1, lista.get(j));
            j--;
        }
        lista.set(j + 1, chave);
    }
}

// 4. MERGE SORT - Divisão e Conquista (Estável)
public static <T extends Comparable<T>> void mergeSort(List<T> lista) {
    if (lista.size() <= 1) return;
    int meio = lista.size() / 2;
    List<T> esq = new ArrayList<>(lista.subList(0, meio));
    List<T> dir = new ArrayList<>(lista.subList(meio, lista.size()));
    mergeSort(esq); mergeSort(dir);
    intercalar(lista, esq, dir);
}

private static <T extends Comparable<T>> void intercalar(List<T> l, List<T> e, List<T> d) {
    int i = 0, j = 0, k = 0;
    while (i < e.size() && j < d.size()) {
        l.set(k++, (e.get(i).compareTo(d.get(j)) <= 0) ? e.get(i++) : d.get(j++));
    }
    while (i < e.size()) l.set(k++, e.get(i++));
    while (j < d.size()) l.set(k++, d.get(j++));
}

// 5. QUICKSORT - Particionamento em torno do pivô
public static <T extends Comparable<T>> void quickSort(List<T> lista, int ini, int fim) {
    if (ini < fim) {
        int p = particionar(lista, ini, fim);
        quickSort(lista, ini, p - 1);
        quickSort(lista, p + 1, fim);
    }
}

private static <T extends Comparable<T>> int particionar(List<T> l, int ini, int fim) {
    T pivo = l.get(fim); int i = ini - 1;
    for (int j = ini; j < fim; j++) {
        if (l.get(j).compareTo(pivo) <= 0) trocar(l, ++i, j);
    }
    trocar(l, i + 1, fim);
    return i + 1;
}
```

O(N LOG N) 6. HeapSort & Resumo Teórico

```
// 6. HEAPSORT - Ordenação baseada na estrutura Max-Heap
public static <T extends Comparable<T>> void heapSort(List<T> lista) {
    int n = lista.size();
    for (int i = n / 2 - 1; i >= 0; i--) heapify(lista, n, i);
    for (int i = n - 1; i > 0; i--) {
        trocar(lista, 0, i);
        heapify(lista, i, 0);
    }
}

private static <T extends Comparable<T>> void heapify(List<T> l, int n, int i) {
    int maior = i, esq = 2 * i + 1, dir = 2 * i + 2;
    if (esq < n && l.get(esq).compareTo(l.get(maior)) > 0) maior = esq;
    if (dir < n && l.get(dir).compareTo(l.get(maior)) > 0) maior = dir;
    if (maior != i) {
        trocar(l, i, maior);
        heapify(l, n, maior);
    }
}
```

Tabela Comparativa para Provas

Algoritmo	Complexidade (Médio)	Complexidade (Pior)	Estável?	Estratégia Principal
Bubble Sort	$O(n^2)$	$O(n^2)$	Sim	Troca pares adjacentes até flutuar o maior elemento.
Selection Sort	$O(n^2)$	$O(n^2)$	Não	Localiza o menor elemento e move para a posição inicial.
Insertion Sort	$O(n^2)$	$O(n^2)$	Sim	Insere o elemento atual na sublista ordenada à esquerda.
Merge Sort	$O(n \log n)$	$O(n \log n)$	Sim	Divisão e conquista: divide ao meio, ordena e intercala.
QuickSort	$O(n \log n)$	$O(n^2)$	Não	Divisão e conquista: particiona os dados em torno de um pivô.
HeapSort	$O(n \log n)$	$O(n \log n)$	Não	Cria uma árvore Max-Heap e extrai o maior elemento sucessivamente.